

DOCUMENTAÇÃO TÉCNICA

Projeto Paciente+

Alunas:Cristiane Alves, Victoria Luna e Yasmin Vitoria.

O projeto utiliza uma arquitetura Front-end puro (Vanilla JS) com persistência de dados local através do localStorage do navegador. É um sistema de gestão desenhado para clínicas de fisioterapia, focado na separação de responsabilidades entre os módulos JS.

1. Visão Geral e Arquitetura do Sistema

O Paciente+ é uma aplicação Single-Page Application (SPA) simulada, onde a navegação ocorre entre arquivos HTML distintos, mas o carregamento da lógica e a persistência dos dados são feitos exclusivamente no lado do cliente.

Tecnologia Base: HTML5, CSS (inferido), JavaScript (puro).

Persistência de Dados: localStorage. Todos os dados (usuários, funcionários, pacientes) são armazenados no navegador do cliente, não havendo comunicação com um servidor de banco de dados.

Segurança: Implementa um controle de acesso básico através da checagem da chave logado no localStorage.

2. Estrutura de Arquivos e Responsabilidades

A aplicação é modularizada em cinco arquivos HTML (as interfaces) e cinco arquivos JavaScript (a lógica).

2.1. Arquivos de Interface (HTML)

Arquivo	Propósito Principal	Módulos JS Envolvidos
index.html	Tela de Login e Cadastro de Novos Usuários.	login.js
dashboard.html	Página de Menu Principal (Dashboard).	app.js (para proteção)
funcionarios.html	Interface para Cadastro e Listagem de Funcionários.	app.js, funcionarios.js
pacientes.html	Interface para Cadastro de Pacientes.	app.js, paciente.js
tratamentos.html	Listagem e Gerenciamento de Pacientes, Focada em Tratamentos/Sessões.	app.js, paciente.js

2.2. Arquivos de Lógica (JavaScript)

Arquivo	Descrição da Responsabilidade
login.js	Contém a lógica de autenticação. Responsável por fazerLogin() (validar credenciais) e cadastrarUsuario() (registrar novos usuários no sistema).
app.js	Módulo central de controle de acesso. Contém funções como proteger() (middleware de autenticação) e logout() (encerrar sessão), garantindo que apenas usuários logados acessem as páginas internas.
funcionarios.js	Gerencia o CRUD (Criar, Ler, Atualizar, Deletar) de Funcionários. Contém funções para capturar dados do formulário de funcionário, salvar no localStorage e renderizar a lista na interface.
paciente.js	Gerencia o CRUD de Pacientes. Responsável por cadastrarPaciente(), listarPacientes() e, de forma implícita, gerenciar o histórico de tratamentos.
sistema.js	Biblioteca de Funções Auxiliares. Contém utilitários genéricos, como funções para manipular o DOM, exibir mensagens de feedback (alertas) e a lógica para carregar e salvar dados de forma genérica no localStorage.

3. Modelo de Dados

O sistema utiliza o localStorage para armazenar objetos JSON que representam as principais entidades do negócio.

Chave (localStorage)	Estrutura de Dados	Finalidade				
usuarios	Array<Object {username, password}>	Credenciais para acesso ao sistema.				
logado	Object {username, ...}	Armazena a sessão ativa. A presença desta chave indica que o usuário está autenticado.				
funcionarios	Array<Object {id, nome, cpf,.crm, ...}>	Registros de todos os colaboradores, incluindo fisioterapeutas (CRM/CREFITO inferido).				
pacientes	Array<Object {id, nome, cpf, dt_nasc, endereco, ...}>	Registros dos pacientes da clínica, incluindo dados pessoais e clínicos iniciais.				

4. Fluxo de Execução e Funções Chave

A. Fluxo de Autenticação

O usuário acessa index.html.

Ao submeter o formulário, a função fazerLogin() em login.js é acionada.

fazerLogin() verifica se o par username/password existe no array usuarios.

Se for válido, o objeto do usuário é salvo na chave logado e o usuário é redirecionado para dashboard.html.

B. Proteção de Rota (Middleware)

Ao carregar qualquer página interna (ex: funcionarios.html), a função proteger() em app.js é executada (via event listener ou onload).

proteger() verifica a existência da chave logado.

Se a chave não existir ou estiver vazia, o usuário é imediatamente redirecionado para index.html (tela de login).

C. Gerenciamento de Funcionários

Em funcionarios.html, a submissão do formulário aciona cadastrarFuncionario() em funcionarios.js.

Esta função lê o localStorage para obter o array funcionarios, adiciona o novo objeto (com um ID único gerado) e salva o array atualizado.

A função listarFuncionarios() é chamada para buscar os dados atualizados e exibi-los na tabela, utilizando funções de renderização de sistema.js.

A remoção é tratada por removerFuncionario(id) que filtra o array funcionarios e ressalva no localStorage.

D. Gerenciamento de Pacientes e Tratamentos

Em pacientes.html, a função cadastrarPaciente() em paciente.js executa o processo de salvar um novo registro na chave pacientes.

A página tratamentos.html é responsável por listar os pacientes, provavelmente usando listarPacientes() (de paciente.js) e renderizando as informações focadas no tratamento em andamento (sessões, evolução, etc.).

Funções adicionais em paciente.js (não explicitadas, mas necessárias) devem manipular a lista pacientes para registrar a evolução clínica e o controle de sessões de fisioterapia.